Operating Systems

Project Phase 2 Report

Student 1: Sihyun Kim
NetID: sk11019

Student 2: *Shota Matsumoto*
NetID: *sm11745*

0 Contents

1 Architecture and Design **2**
- 1.1 System Overview . . . 2
- 1.2 File Structure . . . 2
- 1.3 Key Design Decisions . . . 2

2 Implementation Highlights **3**
- 2.1 `execute_and_capture()` — Bridging Phase 1 and the Socket . . . 3
- 2.2 Server Main Loop . . . 3
- 2.3 Client . . . 4
- 2.4 Error Handling . . . 6
- 2.5 Server Log Format . . . 6

3 Execution Instructions **6**
- 3.1 Compilation . . . 6
- 3.2 Running the Server . . . 7
- 3.3 Running the Client . . . 7
- 3.4 Running the Local Shell (Phase 1) . . . 7

4 Testing **7**
- 4.1 Test 1: Basic Single Commands . . . 7
- 4.2 Test 2: Commands with Flags . . . 8
- 4.3 Test 3: Pipes . . . 8
- 4.4 Test 4: Output Redirection . . . 9
- 4.5 Test 5: Append Redirection . . . 9
- 4.6 Test 6: Input Redirection . . . 10
- 4.7 Test 7: Error Redirection . . . 10
- 4.8 Test 8: Unknown Command . . . 10
- 4.9 Test 9: Exit . . . 10

5 Challenges **11**

6 Division of Tasks **11**

7 References **12**

1 Architecture and Design

1.1 System Overview

Phase 2 extends the Phase 1 local shell into a client-server architecture using TCP socket communication. The user enters commands into the client terminal. The client transmits the command string over a TCP socket to the server. The server parses and executes it using the Phase 1 shell logic, captures all output, and sends the result back to the client for display.

The high-level data flow is:

1. Client presents a `$` prompt and reads user input.
2. Client sends the command string to the server over a TCP socket.
3. Server calls `parse_line()` and `execute_plan()` from Phase 1.
4. Server captures stdout and stderr via an internal pipe into a buffer.
5. Server sends the captured output back to the client over the socket.
6. Client receives and prints the output, then loops back to step 1.

1.2 File Structure

File	Role
`main.c`	Entry point for the standalone local shell (Phase 1)
`parser.c`	Tokenises input into an `ExecutionPlan` struct; handles pipes, redirection, quoted strings
`executor.c`	Executes a command pipeline using `fork/exec/pipe`; defines `free_plan()`
`server.c`	TCP server: accepts connections, parses and executes commands, sends output back
`client.c`	TCP client: presents shell prompt, sends commands, prints server response
`common.h`	Shared type definitions (`Command`, `ExecutionPlan`) and function declarations
`Makefile`	Compiles all targets: `shell`, `server`, `client`

1.3 Key Design Decisions

- **Reuse of Phase 1 logic:** `server.c` directly calls `parse_line()` and `execute_plan()` without modification, ensuring all Phase 1 features (pipes, redirections, quoted arguments) continue to work remotely.
- **Output capture via pipe:** `execute_and_capture()` forks a child process that redirects both `stdout` and `stderr` to a pipe; the parent reads from the pipe to collect the full output as a string.

- **Dynamic buffer growth:** the capture buffer doubles in size with `realloc` whenever it fills up, preventing truncation of large outputs.
- **Single-client blocking loop:** the server handles one client at a time using a blocking `accept()` call, keeping the implementation simple and correct.
- **`SO_REUSEADDR`:** the server socket is configured with `SO_REUSEADDR` so the port is immediately reusable after a restart, avoiding "address already in use" errors during development.

2 Implementation Highlights

2.1 `execute_and_capture()` — Bridging Phase 1 and the Socket

This is the core server function that connects Phase 1's executor to socket communication. It creates a pipe, forks a child, and redirects the child's `stdout` and `stderr` to the write end of the pipe before calling `execute_plan()`. The parent reads all bytes from the read end into a dynamically growing buffer and returns it as a heap-allocated string to be sent to the client.

Listing 1: Output capture skeleton in server.c

```
char *execute_and_capture(ExecutionPlan *plan) {
    int pipefd[2];
    pipe(pipefd);
    pid_t pid = fork();
    if (pid == 0) {
        close(pipefd[0]);
        dup2(pipefd[1], STDOUT_FILENO); // redirect stdout ->
            pipe
        dup2(pipefd[1], STDERR_FILENO); // redirect stderr ->
            pipe
        close(pipefd[1]);
        execute_plan(plan);             // Phase 1 executor
            runs here
        exit(0);
    }
    close(pipefd[1]);
    // parent reads all output from pipe into a growing buffer
    ...
    wait(NULL);
    return output;
}
```

2.2 Server Main Loop

The server creates a TCP socket, binds to port 8080, and enters an `accept()` loop. For each connected client it enters an inner `recv()` loop, processes commands one by one, and sends responses. The connection closes when the client sends `exit` or disconnects.

Listing 2: Server accept loop skeleton

```
while (1) {
    client_fd = accept(server_fd, ...);
    while (1) {
        recv(client_fd, buffer, ...);
        ExecutionPlan *plan = parse_line(buffer);
        char *output = execute_and_capture(plan);
        send(client_fd, output, strlen(output), 0);
        free(output);
    }
    close(client_fd);
}
```

2.3 Client

The client is a straightforward TCP client. It presents a `$` prompt, reads a line from `stdin`, sends it to the server with `send()`, and receives the response in a loop (to handle multi-chunk responses). Typing `exit` sends the command and then terminates the client.

The client creates a TCP socket using the `socket()` function, converts the IP address from a string to binary form, and establishes a connection to the server using `connect()`. We can also provide the IP address through a command-line argument, but if none is provided, it defaults to localhost (`127.0.0.1`).

Listing 3: Socket initialization and connection in client.c

```
// create TCP socket
sock_fd = socket(AF_INET, SOCK_STREAM, 0);
if (sock_fd < 0){
    perror("socket() failed");
    exit(1);
}

// set the address family to IPv4
server_address.sin_family = AF_INET;
// convert the port number to big-endian
server_address.sin_port = htons(PORT);

// convert the IP address into binary form
int result = inet_pton(AF_INET, server_ip, &server_address.
   sin_addr);
if (result <= 0){
    fprintf(stderr, "This is invalid IP address : %s\n",
       server_ip);
    close(sock_fd);
    exit(1);
}

// create the connection with the server
if (connect(sock_fd, (struct sockaddr *)&server_address, sizeof
   (server_address)) < 0){
    perror("connect() failed");
    close(sock_fd);
```

```
    exit(1);
}
```

The `client.c` file also has the capability to send user input to the server through the implementation above. It reads user input using `fgets()`, removes the newline character at the end while ignoring empty lines, and then sends the input using `send()` over the socket connection.

Listing 4: User input and command transmission in client.c

```
// read the user input, send it to the server, and then display
    the response
while (1){
    // display the shell prompt
    printf("$ ");
    fflush(stdout);

    // read the user input from stdin
    if(!fgets(input, sizeof(input), stdin)){
        break;
    }

    // remove the newline character at the end
    input[strcspn(input, "\n")] = 0;

    // skip the empty line
    if (strlen(input) == 0){
        continue;
    }

    // send the user input over the socket
    if (send(sock_fd, input, strlen(input), 0) < 0){
        perror("send() failed");
        break;
    }
}
```

Furthermore, the client enters a loop to receive the entire response from the server. It ensures that the response is displayed immediately, and the loop breaks when `recv()` returns fewer than 4096 bytes.

Listing 5: Response reception loop in client.c

```
// receive the response from the server using while loop
while (1){
    // fill the response buffer with 0 for cleaning
    memset(response, 0, sizeOfBuffer);
    // receive the response from the server
    int num_bytes_received = recv(sock_fd, response,
       sizeOfBuffer - 1, 0);

    // error handling
```

```
    if (num_bytes_received < 0){
        perror("recv() failed");
        break;
    }

    // exit if the server has disconnected
    if (num_bytes_received == 0){
        printf("Server has disconnected.\n");
        close(sock_fd);
        exit(1);
    }

    // null terminate to treat response as C string
    response[num_bytes_received] = '\0';
    printf("%s", response);
    fflush(stdout);

    // break if we received all the data
    if (num_bytes_received < sizeOfBuffer - 1){
        break;
    }
}
```

2.4 Error Handling

- `parse_line()` returns `NULL` on malformed input (leading/trailing/consecutive pipes, missing redirection targets); the server sends `"Error: invalid command."` to the client.
- `fork()` and `pipe()` failures inside `execute_and_capture()` return descriptive error strings instead of crashing the server.
- `recv()` returning 0 is treated as a clean client disconnect; negative values trigger `perror` and the connection is closed.
- The server always sends at least a newline (`"\n"`) so the client never blocks indefinitely.
- Command-not-found errors from `execvp()` are written to `stderr`, captured by the pipe, and relayed to the client.

2.5 Server Log Format

The server prints structured log messages matching the format specified in the project handout:

3 Execution Instructions

3.1 Compilation

From the project directory on the remote Linux server:

Tag	Meaning
[INFO]	Server start, client connect/disconnect events
[RECEIVED]	Command string received from the client
[EXECUTING]	Command is about to be parsed and executed
[OUTPUT]	Output being sent back to the client
[ERROR]	Output contains error indicators (command not found, etc.)

```
make          # builds: shell, server, client
make clean    # removes compiled binaries
```

3.2 Running the Server

```
./server
```

The server prints `[INFO] Server started, waiting for client connections...` and blocks on port 8080.

3.3 Running the Client

```
./client                # connects to localhost (127.0.0.1)
./client <IP_ADDRESS> # connects to a remote server
```

The client presents a `$` prompt. Type any Phase 1 command and press Enter. Type `exit` to quit.

3.4 Running the Local Shell (Phase 1)

```
./shell
```

4 Testing

All tests were run on the NYU remote Linux server with the server and client in separate terminal sessions.

4.1 Test 1: Basic Single Commands

We tested basic single commands, such as `ls`, `pwd`, and `echo hello world`. As you can see in the screenshots, the client sends each command to the server over the socket, correctly displaying the expected outcomes under `$`. Additionally, the server correctly displayed each step of the communication, including `[RECEIVED]`, `[EXECUTING]`, and `[OUTPUT]`. All three commands showed the expected outputs as well, so we can confirm the socket communication is working properly with basic single commands.

```
$ ls
a.txt
client
client.c
client.o
common.h
executor.c
executor.o
input
main.c
main.o
Makefile
myshell
myshell.dSYM
parser.c
parser.o
Phase1_report.pdf
server
server.c
server.o
$ pwd
/home/sm11745/OS-Project
$ echo hello world
hello world
$
$
```

```
sm11745@DCLAP-V1525-CSD:~/OS-Project$ ./server
[INFO] Server started, waiting for client connections...
[INFO] Client connected.
[RECEIVED] Received command: "ls" from client.
[EXECUTING] Executing command: "ls"
[OUTPUT] Sending output to client:
a.txt
client
client.c
client.o
common.h
executor.c
executor.o
input
main.c
main.o
Makefile
myshell
myshell.dSYM
parser.c
parser.o
Phase1_report.pdf
server
server.c
server.o

[RECEIVED] Received command: "pwd" from client.
[EXECUTING] Executing command: "pwd"
[OUTPUT] Sending output to client:
/home/sm11745/OS-Project

[RECEIVED] Received command: "echo hello world" from client.
[EXECUTING] Executing command: "echo hello world"
[OUTPUT] Sending output to client:
hello world
```

4.2 Test 2: Commands with Flags

We tested commands with flags, such as `ls -l` and `ps aux`. The client sent each command properly to the server and displayed the response sent over the socket. As you can see in the screenshot, the server correctly displayed `[RECEIVED]`, `[EXECUTING]`, and `[OUTPUT]` for each command. Both commands produced the expected output, and this confirms that the socket communication works properly.

```
$ ls -l
total 1280
-rw-r--r-- 1 sm11745 sm11745      24 Apr  4 06:41 a.txt
-rwxrwxr-x 1 sm11745 sm11745   21512 Apr  4 06:45 client
-rw-r--r-- 1 sm11745 sm11745    3220 Apr  4 06:41 client.c
-rw-rw-r-- 1 sm11745 sm11745   12840 Apr  4 06:45 client.o
-rw-r--r-- 1 sm11745 sm11745     949 Apr  4 06:41 common.h
-rw-r--r-- 1 sm11745 sm11745    4285 Apr  4 06:40 executor.c
-rw-rw-r-- 1 sm11745 sm11745   13312 Apr  4 06:45 executor.o
-rw-r--r-- 1 sm11745 sm11745      22 Apr  4 06:40 input
-rw-r--r-- 1 sm11745 sm11745    1299 Apr  4 06:41 main.c
-rw-rw-r-- 1 sm11745 sm11745    9096 Apr  4 06:45 main.o
-rw-r--r-- 1 sm11745 sm11745     932 Apr  4 06:40 Makefile
-rwxrwxr-x 1 sm11745 sm11745   31608 Apr  4 06:45 myshell
drwxr-xr-x 3 sm11745 sm11745    4096 Apr  4 06:40 myshell.dSYM
-rw-r--r-- 1 sm11745 sm11745    8143 Apr  4 06:43 parser.c
-rw-rw-r-- 1 sm11745 sm11745   15168 Apr  4 06:45 parser.o
-rw-r--r-- 1 sm11745 sm11745 1076834 Apr  4 06:40 Phase1_report.pdf
-rwxrwxr-x 1 sm11745 sm11745   38192 Apr  4 06:45 server
-rw-r--r-- 1 sm11745 sm11745    5541 Apr  4 06:40 server.c
-rw-rw-r-- 1 sm11745 sm11745   17008 Apr  4 06:45 server.o
```

```
[RECEIVED] Received command: "ls -l" from client.
[EXECUTING] Executing command: "ls -l"
[OUTPUT] Sending output to client:
total 1280
-rw-r--r-- 1 sm11745 sm11745      24 Apr  4 06:41 a.txt
-rwxrwxr-x 1 sm11745 sm11745   21512 Apr  4 06:45 client
-rw-r--r-- 1 sm11745 sm11745    3220 Apr  4 06:41 client.c
-rw-rw-r-- 1 sm11745 sm11745   12840 Apr  4 06:45 client.o
-rw-r--r-- 1 sm11745 sm11745     949 Apr  4 06:41 common.h
-rw-r--r-- 1 sm11745 sm11745    4285 Apr  4 06:40 executor.c
-rw-rw-r-- 1 sm11745 sm11745   13312 Apr  4 06:45 executor.o
-rw-r--r-- 1 sm11745 sm11745      22 Apr  4 06:40 input
-rw-r--r-- 1 sm11745 sm11745    1299 Apr  4 06:41 main.c
-rw-rw-r-- 1 sm11745 sm11745    9096 Apr  4 06:45 main.o
-rw-r--r-- 1 sm11745 sm11745     932 Apr  4 06:40 Makefile
-rwxrwxr-x 1 sm11745 sm11745   31608 Apr  4 06:45 myshell
drwxr-xr-x 3 sm11745 sm11745    4096 Apr  4 06:40 myshell.dSYM
-rw-r--r-- 1 sm11745 sm11745    8143 Apr  4 06:43 parser.c
-rw-rw-r-- 1 sm11745 sm11745   15168 Apr  4 06:45 parser.o
-rw-r--r-- 1 sm11745 sm11745 1076834 Apr  4 06:40 Phase1_report.pdf
-rwxrwxr-x 1 sm11745 sm11745   38192 Apr  4 06:45 server
-rw-r--r-- 1 sm11745 sm11745    5541 Apr  4 06:40 server.c
-rw-rw-r-- 1 sm11745 sm11745   17008 Apr  4 06:45 server.o
```

4.3 Test 3: Pipes

We tested the commands including pipes, such as `ls | grep .c` and `ls -l | grep .c | wc -l`. The client sent the entire pipe command as a single string to the server and displayed the returned response sent over the socket. We can confirm that the server also received the whole pipe command as well and executed them properly. Since the commands produced the expected output, we can confirm that socket communication works perfectly fine with pipes too.

```
$ ls | grep .c
client.c
executor.c
executor.o
main.c
parser.c
server.c
$ ls -l | grep .c | wc -l
9
```

```
[RECEIVED] Received command: "ls | grep .c" from client.
[EXECUTING] Executing command: "ls | grep .c"
[OUTPUT] Sending output to client:
client.c
executor.c
executor.o
main.c
parser.c
server.c

[RECEIVED] Received command: "ls -l | grep .c | wc -l" from client.
[EXECUTING] Executing command: "ls -l | grep .c | wc -l"
[OUTPUT] Sending output to client:
9
```

4.4 Test 4: Output Redirection

We tested the commands with output redirection, including `ls > output.txt` and `cat output.txt`. When we execute `ls > output.txt`, nothing gets returned so the server sends the newline character to the client to show the `$` prompt immediately, allowing users to enter the next commands. Since we got the expected output, we can confirm that the socket communication works properly.

```
$ ls > output.txt

$ cat output.txt
a.txt
client
client.c
client.o
common.h
executor.c
executor.o
input
main.c
main.o
Makefile
myshell
myshell.dSYM
output.txt
parser.c
parser.o
Phase1_report.pdf
server
server.c
server.o
$
```

```
[RECEIVED] Received command: "ls > output.txt" from client.
[EXECUTING] Executing command: "ls > output.txt"
[OUTPUT] Sending output to client:

[RECEIVED] Received command: "cat output.txt" from client.
[EXECUTING] Executing command: "cat output.txt"
[OUTPUT] Sending output to client:
a.txt
client
client.c
client.o
common.h
executor.c
executor.o
input
main.c
main.o
Makefile
myshell
myshell.dSYM
output.txt
parser.c
parser.o
Phase1_report.pdf
server
server.c
server.o
```

4.5 Test 5: Append Redirection

We tested append redirection, including `echo hello >> output.txt` and `cat output.txt`. Likewise, `echo hello >> output.txt` returned nothing so we send the newline character to the client to immediately show `$` prompt. Both commands produced the expected output, confirming that socket communication works properly.

```
$ echo hello >> output.txt

$
$ cat output.txt
a.txt
client
client.c
client.o
common.h
executor.c
executor.o
input
main.c
main.o
Makefile
myshell
myshell.dSYM
output.txt
parser.c
parser.o
Phase1_report.pdf
server
server.c
server.o
hello
$
```

```
[RECEIVED] Received command: "echo hello >> output.txt" from client.
[EXECUTING] Executing command: "echo hello >> output.txt"
[OUTPUT] Sending output to client:

[RECEIVED] Received command: "cat output.txt" from client.
[EXECUTING] Executing command: "cat output.txt"
[OUTPUT] Sending output to client:
a.txt
client
client.c
client.o
common.h
executor.c
executor.o
input
main.c
main.o
Makefile
myshell
myshell.dSYM
output.txt
parser.c
parser.o
Phase1_report.pdf
server
server.c
server.o
hello
```

4.6 Test 6: Input Redirection

We tested input redirection. The server correctly sent the full content of `output.txt` to the client. Since we got the expected output, we can confirm the socket communication with input redirection works properly.

```
$ cat < output.txt
a.txt
client
client.c
client.o
common.h
executor.c
executor.o
input
main.c
main.o
Makefile
myshell
myshell.dSYM
output.txt
parser.c
parser.o
Phase1_report.pdf
server
server.c
server.o
hello
$
```

```
[RECEIVED] Received command: "cat < output.txt" from client.
[EXECUTING] Executing command: "cat < output.txt"
[OUTPUT] Sending output to client:
a.txt
client
client.c
client.o
common.h
executor.c
executor.o
input
main.c
main.o
Makefile
myshell
myshell.dSYM
output.txt
parser.c
parser.o
Phase1_report.pdf
server
server.c
server.o
hello
```

4.7 Test 7: Error Redirection

We tested error redirection. As you can see in the screenshot we can confirm that the error message from `ls` was redirected to `error.log` on the server. Since nothing is returned, we send the newline character to the client to immediately show the `$` prompt. We ran `cat error.log` and confirmed that the error was correctly captured.

```
$ ls /nonexistent 2> error.log

$
$ cat error.log
ls: cannot access '/nonexistent': No such file or directory
$
```

```
[RECEIVED] Received command: "ls /nonexistent 2> error.log" from client.
[EXECUTING] Executing command: "ls /nonexistent 2> error.log"
[OUTPUT] Sending output to client:

[RECEIVED] Received command: "cat error.log" from client.
[EXECUTING] Executing command: "cat error.log"
[OUTPUT] Sending output to client:
ls: cannot access '/nonexistent': No such file or directory
```

4.8 Test 8: Unknown Command

We tested the unknown command. As you can see in the screenshot, the server tried to execute it but then since the command does not exist on the server, it sent the error message to the client.

```
[RECEIVED] Received command: "unknowncmd" from client.
[EXECUTING] Executing command: "unknowncmd"
[OUTPUT] Sending output to client:
unknowncmd: Command not found.
```

```
$ unknowncmd
unknowncmd: Command not found.
$
```

4.9 Test 9: Exit

Finally, we tested the exit. As you can see in the screenshot, the server correctly closed the connection with the client.

```
$ exit
sm11745@DCLAP-V1525-CSD:~/OS-Project$
```

```
[RECEIVED] Received command: "exit" from client.
[INFO] Client requested exit. Closing connection.
```

5 Challenges

In terms of `server.c`, the most significant challenge was capturing the output of `execute_plan()` so that it could be sent back to the client over the socket. Since Phase 1's `execute_plan()` writes directly to the process's `stdout` and `stderr` file descriptors, there was no straightforward way to intercept this output. To solve this, I introduced an additional `fork()` inside `execute_and_capture()`: the child process uses `dup2()` to redirect both `stdout` and `stderr` into the write end of a pipe before calling `execute_plan()`, while the parent reads all output from the read end into a dynamically allocated buffer. This approach allowed the core logic of `execute_plan()` to be reused in the server without requiring structural changes to its pipeline or redirection handling.

Another challenge was handling output of unpredictable size. Commands like `cat` on a large file could produce far more output than a fixed buffer could hold. To address this, I implemented a doubling strategy using `realloc()`: whenever the buffer fills up, its capacity is doubled and reading continues, ensuring that no output is silently truncated regardless of size.

In terms of `client.c`, socket connection initialization was straightforward since I had experience with it in Computer Networks. However, I struggled to determine when the server had finished sending the full response. The issue was that TCP sockets do not have a built-in function to indicate whether the server has finished sending data. If we call `recv()` only once, we might miss a large amount of data; at the same time, if we keep calling it in a loop, we may block while waiting for additional data that does not exist. Therefore, I added an `if` statement at the end to ensure that if the number of bytes received is less than 4096, we can assume that we have received the full response.

Additionally, while testing use cases, we realized that we had not accounted for the case where nothing is received from the server. To address this issue, we added an `if` statement to check whether the size of the returned output is 0. If it is 0, we return a newline character, prompting the client to immediately display the `$` prompt and allow users to enter the next commands.

6 Division of Tasks

Shota Matsumoto was mainly responsible for developing `client.c`, ensuring that the client initiates the socket connection, accepts user input in a loop, and displays the response. Sihyun Kim implemented `server.c`, ensuring that the socket is properly set up, commands are executed correctly, their output is captured, and server communication runs continuously in a loop. Both members contributed equally to testing and debugging to ensure the system works properly on the NYU remote Linux server.

Member	**Responsibilities**
Sihyun Kim	*server.c,debugging based on the feedback, testing, report writing*
Shota Matsumoto	*client.c, debugging based on the feedback, testing, report writing*

7 References

1. Course Lecture and Lab Slides, *Operating Systems*, NYU Abu Dhabi, Spring 2026. Used as the primary reference for the implementation
2. Anthropic, *Claude AI Assistant*, claude.ai, 2026. Used for debugging assistance and code review during development.